

Pharmacy Microservices - Testing Quick Reference Card

30-Second Setup

```
# Terminal 1
cd C:\RN\Microservice-API-Gateway && npm run dev

# Terminal 2
cd C:\RN\PHRMA-PRODUCTION-APP-BACKEND-MAIN && npm run dev

# Terminal 3
cd C:\RN\PHRMA-PRODUCTION-APP-BACKEND-MAIN-2 && npm run dev

# Postman: Import Pharmacy_API_Collection.postman_collection.json
```

Authentication

Token Header:

```
Authorization: Bearer {{auth_token}}
```

Generate Token (Node.js):

```
node -e "const jwt = require('jsonwebtoken');
console.log(jwt.sign({_id:'507f1f77bcf86cd799439011', email:'test@example.com',
role:'customer'}, 'SECRET', {expiresIn:'120d'}))"
```

Service Endpoints

Service	Port	Check
API Gateway	3000	<code>curl http://localhost:3000</code>
OCR Service	5000	<code>curl http://localhost:5000</code>
Store Service	5000	<code>curl http://localhost:5000</code>
gRPC	50051	<code>nc -zv localhost 50051</code>

Core API Routes

Prescription

```
POST /api/prescription/upload (Auth ✓)
POST /api/prescription/upload-stream (Auth ✓)
```

Search

```
POST /api/search (Auth ✓)
GET /api/search/global (Auth X)
```

Bucket

```
GET /api/bucket (Auth ✓)
POST /api/bucket/add (Auth ✓)
POST /api/bucket/remove (Auth ✓)
```

Store

```
GET /v2/medicine-store/all (Auth X)
GET /v2/medicine-store/:id (Auth X)
GET /v2/medicine-store/:id/items (Auth ✓)
GET /v2/medicine-store/:id/items/search (Auth ✓)
GET /v2/medicine-store/:id/items/available (Auth ✓)
GET /v2/medicine-store/:id/review-stats (Auth X)
GET /v2/medicine-store/:id/hours (Auth X)
GET /v2/medicine-store/:id/inventory-status (Auth X)
POST /v2/medicine-store/:id/review (Auth ✓)
```

Quick API Calls (CURL)

Upload Prescription

```
curl -X POST "http://localhost:3000/api/prescription/upload" \
  -H "Authorization: Bearer $TOKEN" \
  -F "prescription=@prescription.jpg"
```

Search Medicines

```
curl -X POST "http://localhost:3000/api/search" \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{"prescriptionId":"id_here"}'
```

Get All Stores

```
curl "http://localhost:3000/v2/medicine-store/all"
```

Add to Bucket

```
curl -X POST "http://localhost:3000/api/bucket/add" \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{"prescriptionId":"id","store":  
{"storeId":"id","storeName":"name","medicines":[]}}'
```

☒ Test Sequence

1. **Get All Stores** (No auth needed)

```
curl http://localhost:3000/v2/medicine-store/all
```

2. **Upload Prescription** (Auth needed)

```
curl -X POST http://localhost:3000/api/prescription/upload \  
-H "Authorization: Bearer $TOKEN" \  
-F "prescription=@file.jpg"
```

🔗 Save _id → use as {{prescription_id}}

3. **Search Medicines** (Auth needed)

```
curl -X POST http://localhost:3000/api/search \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{"prescriptionId":"{{prescription_id}}"'
```

4. Add to Bucket (Auth needed)

```
curl -X POST http://localhost:3000/api/bucket/add \
-H "Authorization: Bearer $TOKEN" \
-d '{"prescriptionId":"id","store":{"...}}'
```

5. Get Bucket (Auth needed)

```
curl http://localhost:3000/api/bucket \
-H "Authorization: Bearer $TOKEN"
```

Performance Targets

Operation	Target
Upload Prescription	< 25s
Search by Prescription	< 5s
Get All Stores	< 3s
Get Bucket	< 1s
Add to Bucket	< 1s

Common Errors & Fixes

Error	Cause	Fix
401 Unauthorized	Invalid/missing JWT	Check Authorization header
404 Not Found	Resource doesn't exist	Verify resource ID in database
Connection Refused	Service not running	Check if service is started
Timeout	Server overloaded	Increase timeout, reduce load
CORS Error	Cross-origin request blocked	Check CORS configuration
413 File Too Large	Image > 5MB	Compress image
400 Bad Request	Malformed JSON	Check request format

Debugging Checklist

- ☐ All services running? (`curl http://localhost:3000`)
- ☐ MongoDB running? (Can services connect?)

- ☐ JWT token valid? (Not expired)
- ☐ Request format correct? (JSON, headers)
- ☐ Path parameters correct? (IDs exist)
- ☐ Content-Type header set? (application/json for POST)
- ☐ Authorization header set? (For protected routes)
- ☐ Network connection stable?

Environment Variables (Postman)

```
gateway_url = http://localhost:3000
ocr_service_url = http://localhost:5000
store_service_url = http://localhost:5000
auth_token = [YOUR_JWT_TOKEN]
user_id = 507f1f77bcf86cd799439011
prescription_id = [SAVE_FROM_UPLOAD_RESPONSE]
store_id = 507f1f77bcf86cd799439012
medicine_id = 507f191e810c19729de860e1
```

Documentation Files

File	Purpose	Time
POSTMAN_API_TESTING_GUIDE.md	Complete API reference	20 min read
Pharmacy_API_Collection.postman_collection.json	Import to Postman	2 min import
TESTING_CURL_REFERENCE.md	CURL commands	15 min read
TESTING_CHECKLIST.md	27 test cases	2 hours tests
API_TESTING_INDEX.md	Navigation & overview	10 min read

Quick Testing Scenarios

Scenario 1: Just Verify Connectivity (5 min)

```
curl http://localhost:3000
curl http://localhost:5000
curl http://localhost:3000/v2/medicine-store/all
# If all return 200 OK → Ready
```

Scenario 2: Test Public APIs Only (20 min)

1. Get All Stores
2. Get Store Details

3. Global Medicine Search
4. Get Review Stats

Scenario 3: Test Full User Flow (45 min)

1. Upload Prescription
2. Search Medicines
3. Add to Bucket
4. View Bucket
5. Give Review

Scenario 4: Load Testing (30 min)

```
# Send 50 concurrent requests
for i in {1..50}; do
  curl http://localhost:3000/v2/medicine-store/all &
done
wait
echo "All requests completed"
```

Security Checks

- ☐ SQL injection attempts rejected?
- ☐ XSS attempts in comments sanitized?
- ☐ File uploads validated?
- ☐ JWT expiration enforced?
- ☐ Invalid tokens rejected?
- ☐ Rate limiting working?

Important Ports

```
3000 → API Gateway
5000 → OCR Service (HTTP)
5000 → Store Service (HTTP)
50051 → Store Service (gRPC)
27017 → MongoDB (if local)
6379 → Redis (if local)
9200 → Elasticsearch (if local)
```

Response Status Codes

- ✓ 200 OK
- ✓ 201 Created
- ✗ 400 Bad Request
- ✗ 401 Unauthorized
- ✗ 403 Forbidden
- ✗ 404 Not Found
- ✗ 500 Server Error

Common Payloads

Upload Prescription

```
Form: multipart/form-data
Field: prescription (file)
```

Search Medicines

```
{
  "prescriptionId": "...",
  "filters": {
    "maxPrice": 500,
    "maxDistance": 10
  }
}
```

Add to Bucket

```
{
  "prescriptionId": "...",
  "store": {
    "storeId": "...",
    "storeName": "City Pharmacy",
    "medicines": [
      {"medicineId": "...", "quantity": 2, "price": 45}
    ]
  }
}
```

Need Help?

1. **Check documentation:** POSTMAN_API_TESTING_GUIDE.md

2. **See test examples:** Pharmacy_API_Collection.postman_collection.json
3. **Run CURL commands:** TESTING_CURL_REFERENCE.md
4. **Use test checklist:** TESTING_CHECKLIST.md
5. **Full navigation:** API_TESTING_INDEX.md

Postman Tips

1. **Reuse variables:** `{{variable_name}}` in URLs and bodies
2. **Auto-save responses:** Check request history
3. **Test scripts:** Pre-request tab → Setup auth automatically
4. **Save results:** Export collection after changes
5. **Share collection:** Export → Send to team

Production Readiness

Before deploying to production:

- ☐ All 16 APIs tested and working
- ☐ Performance targets met
- ☐ Security tests passed
- ☐ Error handling validated
- ☐ Database backups verified
- ☐ Monitoring/logging configured
- ☐ Documentation updated
- ☐ Team trained on APIs

Today's Testing Plan

```
[ ] 09:00 - Service health check (5 min)
[ ] 09:05 - Setup Postman (5 min)
[ ] 09:10 - Public APIs test (20 min)
[ ] 09:30 - Authentication test (10 min)
[ ] 09:40 - Prescription flow (30 min)
[ ] 10:10 - Store operations (20 min)
[ ] 10:30 - Integration tests (15 min)
[ ] 10:45 - Performance tests (15 min)
[ ] 11:00 - Security tests (10 min)
[ ] 11:10 - Issues documentation (10 min)
[ ] 11:20 - Final sign-off
```

Keep this card handy while testing!

Last Updated: April 30, 2026

